Environmental ML and AI Final Project

Written by: Benjamin Klass

ID number: 346612021

1. Introduction

1.1 Study Area and Scientific Motivation

The Hula Valley is an alluvial valley located in northern Israel, north of the Sea of Galilee. It plays a critical role in regional water provision, national agriculture, and international bird migration. Historically, the valley contained Lake Hula and extensive seasonal swamps, which were drained in the 1950s to enable agricultural development.

The drainage led to significant environmental consequences, including peat soil subsidence and the transport of agricultural pollutants into the Sea of Galilee. These impacts prompted partial ecological restoration efforts, resulting in the creation of Agamon Hula. Today, the Hula Valley represents a transformed and partially restored wetland–agricultural landscape with decades of available environmental and spatial data, making it a valuable site for land-cover analysis.

In the context of increasing water scarcity across Israel and the Mediterranean region, accurate land-cover information around freshwater systems is essential for environmental monitoring, habitat conservation, agricultural management, and water-resource planning.

1.2 Remote Sensing and Machine Learning Background

Advances in remote sensing have enabled consistent, high-resolution monitoring of land cover, particularly in regions where ground-based observations are limited. More recently, machine learning techniques have facilitated the automated analysis of large volumes of satellite imagery using supervised classification methods.

Among the most widely applied machine learning algorithms for land-cover classification are Random Forest (RF), Support Vector Machine (SVM), and Artificial Neural Networks (ANN). While each has demonstrated strong performance, their effectiveness varies depending on landscape complexity, spectral similarity between classes, and sampling strategies.

1.3 Research Objectives

The objective of this investigation is to classify land cover in the Hula Valley using supervised machine learning techniques and to compare the performance of RF, SVM, and ANN classifiers. Model performance is evaluated using confusion matrices and standard accuracy metrics to assess their suitability for mapping a complex wetland–agricultural environment.

2. Methodology

2.1 Data Sources and Pre-Processing

Sentinel-2 multispectral imagery from the Copernicus Sentinel missions was used, covering the period from 1 April to 31 October 2024. The selected bands were B2 (Blue), B3 (Green), B4 (Red), B8 (NIR), B11 (SWIR-1), and B12 (SWIR-2).

Cloud masking was applied to remove clouds, cloud shadows, and snow contamination. Four spectral indices were derived: NDVI, NDWI, MNDWI, and NDBI. A median composite was generated from all cloud-masked images, and the selected bands and indices were combined into a single feature image.

2.2 Ground Truth and Sampling Strategy

The Google Dynamic World dataset was used as a reference for training labels. Each pixel was assigned its most frequently occurring land-cover class over the study period (modal class). Eight classes were used: Water, Trees, Grass, Flooded Vegetation, Crops, Shrub and Scrub, Built Area, and Bare Ground.

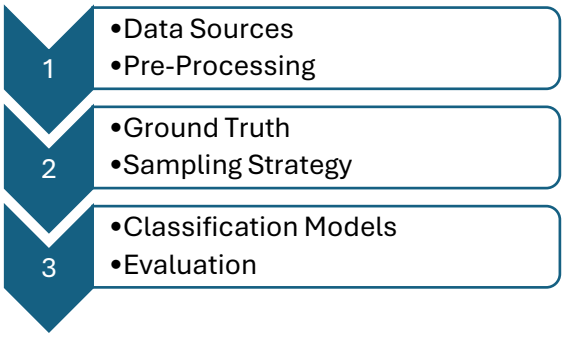
Two sampling approaches were tested. Random sampling resulted in severe class imbalance due to dominance of cropland and vegetation. Stratified sampling was therefore adopted, producing a more balanced class distribution and enabling reliable model training and evaluation.

2.3 Classification Models and Evaluation

Three classifiers were implemented: Random Forest, Support Vector Machine with an RBF kernel, and an Artificial Neural Network. The dataset was split into 70% training and 30% testing samples using a fixed random seed (42).

Model performance was evaluated using confusion matrices, overall accuracy, precision, recall, F1-score, and AUC-ROC. RF and SVM were trained in Google Earth Engine, while ANN training and evaluation were conducted in Python using the exported samples.

2.4 Flowchart



---

3. Results

3.1 Dataset Summary

Table 1. Dataset characteristics used for training and evaluation.

Item	Value
Training samples	8,358
Testing samples	3,642
Number of classes	8 (Dynamic World classes 0–7)
Feature dimension	10 (6 spectral bands + 4 indices)
Spatial resolution	10 m

3.2 Overall Classification Performance

Table 2. Overall performance metrics for GEE and Python models.

Model	Platform	Overall Accuracy	Kappa	Macro F1	Macro AUC
Random Forest	GEE	0.736	0.699	–	–
SVM (RBF)	GEE	0.672	0.625	–	–
Random Forest	Python	0.738	–	0.736	0.956

Notes:

- Kappa is reported only for GEE classifiers.
- Macro F1 and AUC were computed in Python.

3.3 Confusion Matrix (Random Forest – GEE)

Table 3. Confusion matrix for Random Forest classification (rows = reference, columns = predicted).

Ref \ Pred	0	1	2	3	4	5	6	7
0 Water	402	5	2	47	3	3	3	0
1 Trees	3	285	43	50	9	25	23	0
2 Grass	2	62	360	39	5	1	4	0
3 Flooded veg	48	89	42	247	1	9	11	0
4 Crops	1	49	27	3	269	68	53	6
5 Shrub/scrub	0	21	11	0	25	380	14	4
6 Built area	3	36	5	2	29	32	301	17
7 Bare ground	0	0	0	0	3	4	18	438

3.4 Class-wise Accuracy (GEE)

Table 4. Producer's and User's accuracy per class for Random Forest (GEE).

Class	Producer's Accuracy	User's Accuracy
0 Water	0.865	0.867
1 Trees	0.651	0.669
2 Grass	0.761	0.767
3 Flooded vegetation	0.553	0.539
4 Crops	0.565	0.561
5 Shrub & scrub	0.835	0.842
6 Built area	0.708	0.706
7 Bare ground	0.946	0.948

3.5 Python Random Forest: Class-wise Performance

Table 5. Precision, recall, F1-score and support per class (Python RF).

Class	Precision	Recall	F1-score	Support
0 Water	0.88	0.87	0.87	465
1 Trees	0.53	0.67	0.59	438
2 Grass	0.73	0.77	0.75	473
3 Flooded vegetation	0.63	0.54	0.58	447
4 Crops	0.78	0.56	0.65	476
5 Shrub & scrub	0.73	0.84	0.78	455
6 Built area	0.72	0.71	0.71	425
7 Bare ground	0.95	0.95	0.95	463

4. Discussion

Random Forest consistently outperforms SVM in both GEE and Python, indicating its suitability for classifying the complex agricultural–wetland mosaics of the Hula Valley. High AUC values suggest strong class separability, even where confusion between spectrally similar classes persists.

Stratified sampling proves essential for meaningful class-wise evaluation, reducing bias toward dominant land-cover types. Confusion between Crops, Grass, and Flooded Vegetation reflects real ecological overlap driven by seasonal flooding and agricultural practices. Water and Bare Ground exhibit strong separability due to distinct spectral characteristics.

The close agreement between GEE and Python results demonstrates the robustness and reproducibility of the classification pipeline. Limitations include uncertainty in Dynamic World labels and the lack of spatial cross-validation. Additionally, ANN training was conducted in Python due to the absence of native ANN support in GEE.

5. Conclusion

This investigation demonstrates that supervised machine learning, particularly Random Forest, provides an effective approach for land-cover classification in the Hula Valley using Sentinel-2 imagery. Stratified sampling significantly improves minority-class representation and overall model reliability.

The study addresses a gap in machine learning–based land-cover classification in Israel and highlights the challenges of mapping complex wetland–agricultural landscapes using multispectral data. Future research should focus on temporal analysis of land-cover change and on improved characterisation of terrestrial–hydrological interactions through additional data sources.

6. References

Belgiu, M. and Drăguţ, L. (2016) ‘Random forest in remote sensing: A review of applications and future directions’, ISPRS Journal of Photogrammetry and Remote Sensing, 114, pp. 24–31. https://doi.org/10.1016/j.isprsjprs.2016.01.011

BrightChamps (n.d.) Modal class – definition and explanation. Available at: https://brightchamps.com/en-us/math/data/modal-class (Accessed: 24 January 2026).

Congalton, R.G. and Green, K. (2019) Assessing the accuracy of remotely sensed data: Principles and practices. 3rd edn. Boca Raton: CRC Press. https://doi.org/10.1201/9780429052729

Foody, G.M. (2002) 'Status of land cover classification accuracy assessment', Remote Sensing of Environment, 80(1), pp. 185–201. https://doi.org/10.1016/S0034-4257(01)00295-4

GeeksforGeeks (n.d.) Confusion matrix in machine learning. Available at: https://www.geeksforgeeks.org/machine-learning/confusion-matrix-machine-learning/ (Accessed: 24 January 2026).

GeeksforGeeks (n.d.) AUC–ROC curve in machine learning. Available at: https://www.geeksforgeeks.org/machine-learning/auc-roc-curve/ (Accessed: 24 January 2026).

Gophen, M. (2023) 'Biodiversity during pre- and post-Hula Valley (Israel) drainage', Diversity, 15(6), p. 758. https://doi.org/10.3390/d15060758

Hambright, K.D. and Zohary, T. (1998) 'Lake Hula drainage and ecosystem response', Israel Journal of Earth Sciences, 47, pp. 99–109.

Jafarzadeh, H., Mahdianpari, M., Gill, E.W., Brisco, B. and Mohammadimanesh, F. (2022) 'Remote sensing and machine learning tools to support wetland mapping: A meta-analysis of three decades of research', Remote Sensing, 14(23), p. 6104. https://doi.org/10.3390/rs14236104

Landis, J.R. and Koch, G.G. (1977) 'The measurement of observer agreement for categorical data', Biometrics, 33(1), pp. 159–174. https://doi.org/10.2307/2529310

Mahdavi, S., Salehi, B., Granger, J., Amani, M., Brisco, B. and Huang, W. (2018) 'Remote sensing for wetland classification: A comprehensive review', GIScience & Remote Sensing, 55(5), pp. 623–658. https://doi.org/10.1080/15481603.2017.1419602

Maxwell, A.E., Warner, T.A. and Fang, F. (2018) 'Implementation of machine-learning classification in remote sensing: An applied review', International Journal of Remote Sensing, 39(9), pp. 2784–2817. https://doi.org/10.1080/01431161.2018.1433343

Pal, M. and Mather, P.M. (2005) 'Support vector machines for classification in remote sensing', International Journal of Remote Sensing, 26(5), pp. 1007–1011. https://doi.org/10.1080/01431160512331314083

Rana, V.K. and Suryanarayana, T.M.V. (2020) 'Performance evaluation of maximum likelihood, random forest and support vector machine classifiers for land use/land cover mapping using Sentinel-2 data', Remote Sensing Applications: Society and Environment, 19, p. 100351. https://doi.org/10.1016/j.rsase.2020.100351

Rodriguez-Galiano, V.F., Ghimire, B., Rogan, J., Chica-Olmo, M. and Rigol-Sanchez, J.P. (2012) 'An assessment of the effectiveness of a random forest classifier for land-cover classification', ISPRS Journal of Photogrammetry and Remote Sensing, 67, pp. 93–104.

https://doi.org/10.1016/j.isprsjprs.2011.11.002

State of Israel – Ministry of Environmental Protection (2016) The Hula Valley Wetlands Rehabilitation Project. Jerusalem: Government of Israel. Available at: https://www.gov.il (Accessed: 24 January 2026).

Stehman, S.V. (1997) ‘Selecting and interpreting measures of thematic classification accuracy’, Remote Sensing of Environment, 62(1), pp. 77–89. https://doi.org/10.1016/S0034-4257(97)00083-7

Stehman, S.V. and Foody, G.M. (2009) ‘Accuracy assessment’, in Warner, T.A., Nellis, M.D. and Foody, G.M. (eds.) The SAGE handbook of remote sensing. London: SAGE, pp. 297–309.

United States Geological Survey (n.d.) Generating geomedian composites. Available at: https://knowledge.dea.ga.gov.au/notebooks/How_to_guides/Generating_geomedian_composites/ (Accessed: 24 January 2026).

World Resources Institute and Google (n.d.) Dynamic World land use/land cover dataset. Available at: https://www.dynamicworld.app/ (Accessed: 24 January 2026).

Yuan, S., Liang, X., Lin, T., Chen, S., Liu, R., Wang, J., Zhang, H. and Gong, P. (2025) ‘A comprehensive review of remote sensing in wetland classification and mapping’, Remote Sensing Applications: Society and Environment, early online. Preprint available at: https://arxiv.org/abs/2504.10842 (Accessed: 24 January 2026).

---

7. Technical Appendix

7.1 Google Earth Engine Code (JavaScript)

```
/*******************************************************
 Hula Valley Land Cover Classification (Google Earth Engine)
 - Data: Sentinel-2 SR (COPERNICUS/S2_SR_HARMONIZED)
 - Labels: Dynamic World (training proxy; modal label)
 - Models in GEE: Random Forest (tree-based), SVM (RBF)
 - Exports: classified maps + sample table for Python ANN
*******************************************************/

// ======================================================
// 0) USER INPUT: AOI
// ======================================================
// Option A: Draw polygon in the GEE UI and name it "aoi".
// Option B: Import a FeatureCollection and set:
// var aoi = yourImportedFeatureCollection.geometry();

Map.centerObject(aoi, 11);
Map.addLayer(aoi, { color: "red" }, "AOI (Hula Valley)");
```

```
// ========================================================
// 1) DATE RANGE + CLOUD MASK
// ========================================================
var start = "2024-04-01";
var end   = "2024-10-31";

// Sentinel-2 SR (harmonized)
var s2 = ee.ImageCollection("COPERNICUS/S2_SR_HARMONIZED")
  .filterBounds(aoi)
  .filterDate(start, end)
  .filter(ee.Filter.lt("CLOUDY_PIXEL_PERCENTAGE", 30));

// Cloud mask using Scene Classification Layer (SCL)
function maskS2(img) {
  var scl = img.select("SCL");

  // Mask out: cloud shadow(3), clouds(8,9,10), snow(11)
  var mask = scl.neq(3)
    .and(scl.neq(8))
    .and(scl.neq(9))
    .and(scl.neq(10))
    .and(scl.neq(11));

  // Scale reflectance to 0–1 range
  return img.updateMask(mask).divide(10000);
}

s2 = s2.map(maskS2);


// ========================================================
// 2) FEATURE ENGINEERING
// ========================================================
function addIndices(img) {
```

```
  var ndvi  = img.normalizedDifference(["B8",  "B4"]).rename("NDVI");
  var ndwi  = img.normalizedDifference(["B3",  "B8"]).rename("NDWI");  //
McFeeters-style
  var mndwi = img.normalizedDifference(["B3", "B11"]).rename("MNDWI");
  var ndbi  = img.normalizedDifference(["B11", "B8"]).rename("NDBI");

  return img.addBands([ndvi, ndwi, mndwi, ndbi]);
}

// Median composite over the study period
var composite = s2.map(addIndices).median().clip(aoi);

// Feature stack (bands + indices)
var bands = [
  "B2",  "B3",  "B4",  "B8",  "B11", "B12",  // Sentinel-2 bands
  "NDVI","NDWI","MNDWI","NDBI"              // Indices
];

var featureImg = composite.select(bands);

// RGB visualisation (display only; does not affect analysis)
Map.addLayer(
  featureImg,
  { bands: ["B4", "B3", "B2"], min: 0, max: 0.3 },
  "Sentinel-2 RGB"
);


// ======================================================
// 3) TRAINING LABELS (Dynamic World)
// ======================================================
// Dynamic World V1: label classes 0..8
var dw = ee.ImageCollection("GOOGLE/DYNAMICWORLD/V1")
  .filterBounds(aoi)
  .filterDate(start, end);
```

```
// Modal (most frequent) label over the period
var label = dw.select("label").mode().clip(aoi).rename("label");

Map.addLayer(label.randomVisualizer(), {}, "Dynamic World label (mode)");



// ========================================================
// 4) SAMPLE TRAINING DATA
// ========================================================

// --- First attempt (random sampling; kept here for reference) ---
// var samples = featureImg.addBands(label).sample({
//   region: aoi,
//   scale: 10,
//   numPixels: 12000,
//   seed: 42,
//   geometries: true
// });

// --- Second attempt (stratified sampling; used in final workflow) ---
var samples = featureImg.addBands(label).stratifiedSample({
  numPoints: 1500,      // target samples per class (where possible)
  classBand: "label",
  region: aoi,
  scale: 10,
  seed: 42,
  geometries: false
});

// Train/test split (70/30) using a reproducible random column
var withRand = samples.randomColumn("rand", 42);
var train = withRand.filter(ee.Filter.lt("rand", 0.7));
var test  = withRand.filter(ee.Filter.gte("rand", 0.7));

print("Train size", train.size());
```

```
print("Test size", test.size());


// ========================================================
// 5) MODEL 1: RANDOM FOREST (TREE-BASED)
// ========================================================
var rf = ee.Classifier.smileRandomForest({
  numberOfTrees: 200,
  variablesPerSplit: null,
  minLeafPopulation: 1,
  bagFraction: 0.7,
  seed: 42
}).train({
  features: train,
  classProperty: "label",
  inputProperties: bands
});

var rfClass = featureImg.classify(rf).rename("rf_class");
Map.addLayer(rfClass.randomVisualizer(), {}, "RF classification");

// Evaluate RF
var rfTest = test.classify(rf);
var rfCM = rfTest.errorMatrix("label", "classification");

print("RF confusion matrix", rfCM);
print("RF overall accuracy", rfCM.accuracy());
print("RF kappa", rfCM.kappa());
print("RF producers accuracy (per class)", rfCM.producersAccuracy());
print("RF users accuracy (per class)", rfCM.consumersAccuracy());


// ========================================================
// 6) MODEL 2: SUPPORT VECTOR MACHINE (RBF)
// ========================================================
```

```
var svm = ee.Classifier.libsvm({
  kernelType: "RBF",
  gamma: 0.5,
  cost: 10
}).train({
  features: train,
  classProperty: "label",
  inputProperties: bands
});

var svmClass = featureImg.classify(svm).rename("svm_class");
Map.addLayer(svmClass.randomVisualizer(), {}, "SVM classification");

// Evaluate SVM
var svmTest = test.classify(svm);
var svmCM = svmTest.errorMatrix("label", "classification");

print("SVM confusion matrix", svmCM);
print("SVM overall accuracy", svmCM.accuracy());
print("SVM kappa", svmCM.kappa());
print("SVM producers accuracy (per class)", svmCM.producersAccuracy());
print("SVM users accuracy (per class)", svmCM.consumersAccuracy());


// ========================================================
// 7) EXPORTS FOR PYTHON (ANN) + MAPS
// ========================================================

// Export samples as CSV (includes features + label + split column)
Export.table.toDrive({
  collection: withRand.select(bands.concat(["label", "rand"])),
  description: "Hula_S2_DynamicWorld_samples",
  fileFormat: "CSV"
});
```

```
// Export classified maps (GeoTIFF)
Export.image.toDrive({
  image: rfClass.toInt(),
  description: "Hula_RF_classification",
  region: aoi,
  scale: 10,
  maxPixels: 1e13
});

Export.image.toDrive({
  image: svmClass.toInt(),
  description: "Hula_SVM_classification",
  region: aoi,
  scale: 10,
  maxPixels: 1e13
});

// Export the label map (for visuals/reporting)
Export.image.toDrive({
  image: label.toInt(),
  description: "Hula_DW_label_mode",
  region: aoi,
  scale: 10,
  maxPixels: 1e13
});
```

7.2 Python Code

```
"""
Hula Valley Land Cover ML (Python)
- Input: CSV exported from GEE (features + label + rand)
- Models: Random Forest, SVM (RBF), ANN (MLP)
- Evaluation: confusion matrix, OA, producer/user accuracy, F1, AUC (OvR)
- Outputs: confusion matrices + ROC curves (saved as PNG)
"""
```

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve,
    ConfusionMatrixDisplay,
    accuracy_score,
    f1_score,
)
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier


# ========================================================
# 1) LOAD DATA
# ========================================================
csv_path = Path("Hula_S2_DynamicWorld_samples.csv")
df = pd.read_csv(csv_path)

bands = ["B2", "B3", "B4", "B8", "B11", "B12", "NDVI", "NDWI", "MNDWI", "NDBI"]
target = "label"

# Use the same split field exported from GEE:
#   train: rand < 0.7
#   test : rand >= 0.7
train_df = df[df["rand"] < 0.7].copy()
```

```
test_df  = df[df["rand"] >= 0.7].copy()

X_train = train_df[bands].values
y_train = train_df[target].astype(int).values
X_test  = test_df[bands].values
y_test  = test_df[target].astype(int).values

classes = np.unique(df[target].astype(int).values)

print("Train size:", X_train.shape, "Test size:", X_test.shape)
print("Classes:", classes)



# ========================================================
# 2) HELPERS: producer/user accuracy from confusion matrix
# ========================================================
def producer_user_accuracy(cm: np.ndarray):
    """
    Producer's accuracy (recall per class) = diag / reference total
    User's accuracy (precision per class)  = diag / predicted total

    Notes:
    - In remote sensing terminology:
      Producer's accuracy ~ recall (omission error)
      User's accuracy     ~ precision (commission error)
    """
    diag = np.diag(cm).astype(float)

    # Reference totals per class (row sums)
    ref_sum = cm.sum(axis=1).astype(float)

    # Predicted totals per class (column sums)
    pred_sum = cm.sum(axis=0).astype(float)

    producer = np.divide(diag, ref_sum, out=np.zeros_like(diag), where=ref_sum !=
0)
```

```
    user     = np.divide(diag, pred_sum, out=np.zeros_like(diag), where=pred_sum
!= 0)

    return producer, user



def evaluate_model(name, model, X_train, y_train, X_test, y_test, classes):
    """
    Train model, print metrics, save confusion matrix plot,
    compute multiclass AUC (One-vs-Rest).
    """
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    # Core metrics
    cm = confusion_matrix(y_test, y_pred, labels=classes)
    oa = accuracy_score(y_test, y_pred)
    f1_macro = f1_score(y_test, y_pred, average="macro")

    producer, user = producer_user_accuracy(cm)

    print("\n" + "=" * 60)
    print(name)
    print("=" * 60)
    print("Overall accuracy:", round(oa, 4))
    print("F1 (macro):", round(f1_macro, 4))
    print("\nClassification report:\n")
    print(classification_report(y_test, y_pred, labels=classes, zero_division=0))

    print("Producer's accuracy (per class):", np.round(producer, 4))
    print("User's accuracy (per class):", np.round(user, 4))

    # Confusion matrix plot
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=classes)
    fig, ax = plt.subplots(figsize=(7, 6))
    disp.plot(ax=ax, values_format="d", colorbar=False)
```

```
    ax.set_title(f"Confusion Matrix: {name}")
    fig.tight_layout()
    fig.savefig(f"confusion_{name.replace(' ', '_').lower()}.png", dpi=200)
    plt.close(fig)

    # AUC (OvR): requires scores for each class
    y_test_bin = label_binarize(y_test, classes=classes)

    if hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_test)
    else:
        scores = model.decision_function(X_test)

    auc_macro = roc_auc_score(y_test_bin, scores, average="macro",
multi_class="ovr")
    print("AUC (macro, OvR):", round(auc_macro, 4))

    return model, cm, auc_macro



# ========================================================
# 3) DEFINE MODELS
# ========================================================

# Model 1: Tree-based (Random Forest)
rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_leaf=1,
    max_features="sqrt",
    n_jobs=-1,
    random_state=42,
)

# Model 2: SVM (RBF) - scaling + probability for AUC
svm = Pipeline([
```

```
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="rbf", C=10, gamma="scale", probability=True,
random_state=42)),
])

# Model 3: ANN (MLP) - scaling
ann = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPClassifier(
        hidden_layer_sizes=(128, 64),
        activation="relu",
        solver="adam",
        alpha=1e-4,
        learning_rate_init=1e-3,
        max_iter=200,
        random_state=42,
    )),
])



# ========================================================
# 4) TRAIN + EVALUATE
# ========================================================
trained_rf, rf_cm, rf_auc = evaluate_model("Random Forest", rf, X_train, y_train,
X_test, y_test, classes)
trained_svm, svm_cm, svm_auc = evaluate_model("SVM (RBF)", svm, X_train, y_train,
X_test, y_test, classes)
trained_ann, ann_cm, ann_auc = evaluate_model("ANN (MLP)", ann, X_train, y_train,
X_test, y_test, classes)



# ========================================================
# 5) OPTIONAL: ROC CURVES (OvR) for ONE model (example: ANN)
# ========================================================
def plot_roc_ovr(name, model, X_test, y_test, classes):
    y_bin = label_binarize(y_test, classes=classes)
```

```
    if hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_test)
    else:
        scores = model.decision_function(X_test)

    fig, ax = plt.subplots(figsize=(7, 6))
    for i, c in enumerate(classes):
        fpr, tpr, _ = roc_curve(y_bin[:, i], scores[:, i])
        ax.plot(fpr, tpr, label=f"Class {c}")

    ax.plot([0, 1], [0, 1], linestyle="--")
    ax.set_xlabel("False Positive Rate")
    ax.set_ylabel("True Positive Rate")
    ax.set_title(f"ROC Curves (OvR): {name}")
    ax.legend(loc="lower right", fontsize=8)

    fig.tight_layout()
    fig.savefig(f"roc_{name.replace(' ', '_').lower()}.png", dpi=200)
    plt.close(fig)


plot_roc_ovr("ANN (MLP)", trained_ann, X_test, y_test, classes)

print("\nSaved figures: confusion_*.png and roc_*.png")
```